FINAL-YEAR ENGINEERING PROJECT | IMPLEMENTED PROTOTYPE

CareerNavigator AI

Live Job Market Analytics, Career Recommendation, Skill-Gap Analysis and Personalized Learning Roadmap System

PROJECT PRESENTATION AND VIVA GUIDE

A practical explanation of implemented features, data flow, transparent methods, and honest limitations.

Implementation status as at 13 October 2026

Project in one sentence

The application collects a bounded sample of Google Jobs results through SerpApi, cleans and stores postings, measures skill demand in that sample, and compares student-reported skills with curated roles to create explainable learning plans.

How to present it

Distinguish working software behavior from future work. Do not claim a match score is a hiring probability or that a search sample represents the whole job market.

1. Project purpose and current state

CareerNavigator AI helps students explore how their self-reported skills relate to role requirements and a collected sample of job postings. It is periodically refreshed, not continuously real-time.

Area	Implemented project behavior
Data provider	SerpApi Google Jobs; India defaults; bounded pagination; provider key is backend-only.
Backend	FastAPI endpoints for job search/process, analytics, recommendations, gaps, and roadmaps.
Frontend	React + Vite views: Overview, Job Market, Career Match, Skill Gaps, Learning Roadmap, Sources & Project.
Persistence	SQLAlchemy supports PostgreSQL/MySQL; automated tests use SQLite.
Recommendation	Curated role catalog and transparent weighted skill coverage; no trained hiring model.
Configuration	Private .env is ignored by Git; .env.example uses placeholders.

Verified checks
48 backend tests passed at the recorded verification point; frontend lint and production build passed. The build has a non-blocking large Plotly chunk warning. These checks do not establish market accuracy.

2. Architecture and end-to-end flow

The browser sends JSON requests to FastAPI. The backend handles provider credentials, validation, processing, calculations, and database operations. The React frontend never receives the SerpApi key.

Stage	What happens	Visible evidence
1 Collect	Search role keywords and location through SerpApi; page count is bounded.	Provider name, result rows, collection timestamp.
2 Review	Show returned postings before saving.	Title, company, location, date if exact, source link.
3 Clean/save	Normalize, remove duplicates, extract skills, save raw snapshots and cleaned/upserted jobs.	Created/updated counts and data-quality report.
4 Analyze	Query saved records and calculate filters and skill demand.	Unique sample size, source, date range, percentages.
5 Guide	Compare profile with role skills; classify gaps and generate milestones.	Score explanation, gap priorities, weekly plan.

Layer	Technology / responsibility
UI	React, Vite, responsive CSS, Plotly visual components.
API	Python, FastAPI, Pydantic, httpx.
Storage	SQLAlchemy with PostgreSQL/MySQL; SQLite test database.
Reference data	Editable JSON skills, role weights, and curated resource catalogs.

3. Collection, cleaning, and data quality

The system uses an authorized API integration and does not directly scrape job-board websites. Live search is an external request and may use the account quota. A search is preview-only until the user selects Clean & save jobs.

Field / operation	Handling
provider and provider_id	Provider identity and job identifier; used for deduplication when present.
posted_date	Exact parseable source date only; relative dates remain unknown.
collected_at	Retrieval timestamp kept separate from posted_date.
source_url	Provider listing/share/application URL when supplied.
Missing values	Remain unknown; never filled with invented data.
Cleaning	Title/location normalization, duplicate checks, canonical skill dictionary with aliases.
Quality report	Input/accepted/unique counts, duplicate and missing-title counts, invalid salary and missing-field counts.
Storage	Raw JSON snapshots are kept separately from normalized jobs and skill links.

Sample limitation
Collected jobs reflect a provider response to a specific query/location and time. They are not a census or statistically representative measure of all vacancies.

4. Database and market analytics

Tables	Purpose
raw_job_postings; jobs	Raw snapshots and cleaned/upserted job records.
skills; job_skills	Canonical skill definitions and job-skill many-to-many links.
career_roles; career_role_skills	Curated career role requirements, weights, and target skill levels.
student_profiles; student_skills	Schema structures exist; incoming recommendation profiles are not stored by the current flow.
learning_resources	Skill-linked URLs, provider/type, and verification metadata.
learning_roadmaps; roadmap_milestones	Generated weekly plan persistence when database is configured.

Dashboard measures

The dashboard supports filtered unique-posting counts, role/location groups, skill demand, provider names, posting date coverage, and refresh time where the records support those fields. Filters query the saved database; they do not call SerpApi.

```
100 x (unique selected jobs mentioning a skill) / (all unique selected jobs)
```

Salary/currency, structured experience distribution, and historical trends are reported unavailable when reliable comparable data or repeated snapshots do not exist. A single collection cannot establish a trend.

5. Recommendations, gaps, and learning plan

Role profiles are editable JSON definitions. The seeded roles include Data Analyst, Business Intelligence Analyst, Data Scientist, and Data Engineer. Skill weights and levels are transparent inputs.

```
Match score = 100 x matched requirement weight / total role requirement weight
```

A self-reported skill below its target level contributes partial credit (50% of its weight). Responses explain matched, partial, missing, and unknown skills. This is weighted profile overlap, not a probability of getting hired.

Gap state	Meaning	Priority
Possessed	Reported level meets the role target.	No learning gap.
Partial	Skill reported below target level.	Role weight plus observed skill demand when available.
Missing	Skill not entered in the profile.	Role weight plus observed skill demand when available.

The roadmap converts prioritized partial/missing requirements into ordered weekly goals, reasons, practice tasks, mini-projects, and catalog links when available. Roadmaps can be persisted with the database; completion checkboxes are browser-local. Profiles are analyzed but not stored.

6. Frontend walkthrough and live demo

Screen/action	What to demonstrate
Overview	Saved-sample summary, top skills/locations, posting date range, refresh time when available.
Job Market: Search live jobs	Enter role and city; inspect SerpApi results and collection metadata. This makes a provider request.
Job Market: Clean & save	Explicitly save after review; explain duplicate and quality counts; dashboard refreshes from database.
Job Market: filters	Filter stored data by location, role, or skill; explain unique-job denominator.
Career Match	Submit profile and describe weighted overlap, matched skills, partials, missing skills.
Skill Gaps and Roadmap	Show prioritized gaps, weekly tasks, resource catalog, and progress controls.
Sources & Project	Disclose source, periodic refresh, sample and score limitations.

Five-minute story
Problem/objective; collect and review a small sample; explicitly clean and save; explain sample analytics; submit a student profile; finish with gap priorities and a roadmap.

7. Verification, limitations, and viva

Recorded verification: 48 passing backend tests, frontend lint passed, frontend production build passed with a non-blocking Plotly chunk-size warning. A bounded live SerpApi smoke query returned 10 postings in one page. That one request is not a representative market sample or model evaluation.

Question	Answer
Why FastAPI?	Python-native validation, generated API documentation, and a clear frontend/backend boundary.
Why content-based matching?	It is explainable and does not require an unavailable labelled hiring-outcome dataset.
How is skill demand calculated?	Unique filtered postings mentioning a skill divided by all unique filtered postings, times 100.
Is the data continuously real-time?	No. Provider search is user-initiated; the saved market sample changes when a batch is processed.
What does a role score mean?	Overlap with the project's encoded role skills, not a hiring probability.

Limitations and future work

Provider quota and coverage can change; fields and dates may be missing; dictionary skill extraction can miss context; roles/weights are curated assumptions; sample findings cannot be generalized to the full market. Future work may add scheduled comparable snapshots, manually labelled skill-extraction evaluation, accessibility/usability testing, and formal evaluation protocols. No ML accuracy or employment-outcome claim is made.

References: SerpApi Google Jobs API (serpapi.com/google-jobs-api); FastAPI (fastapi.tiangolo.com); React Learn (react.dev/learn); Vite (vite.dev/guide); Plotly (plotly.com); PostgreSQL (postgresql.org/docs). Verify current API terms and quotas.